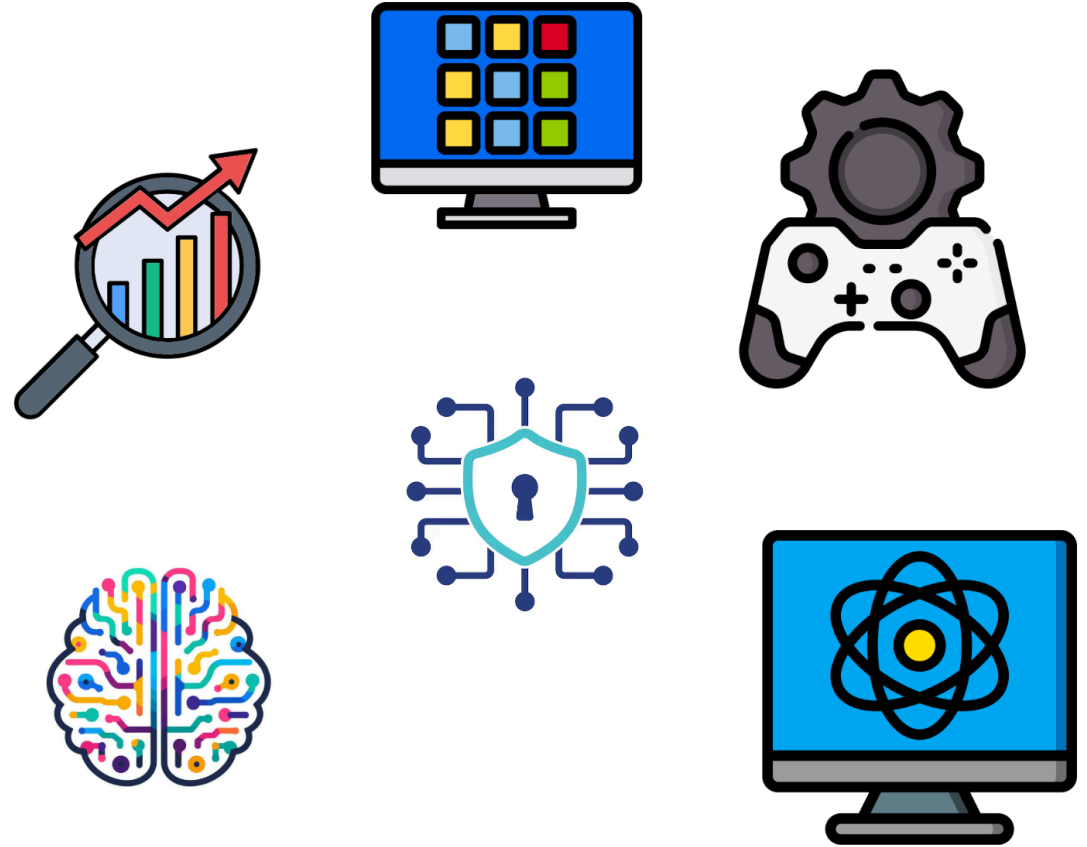


Introduction to Programming in Python

Chapter 1: Why Python? - The applications of the language

1.1. Software Development

- Game Development
- Cyber Security
- Scientific Computing
- Desktop Applications
- Data Analysis
- AI
- Web Development



1.2. Office Work

- Excel Integration
- PDFs
- Databases
- Sending emails



1.3. Long-Term Career Goals

- Web Development
 - Linux
 - HTML, CSS, JAVASCRIPT
 - HTTP, URLS, WEB SERVERS
- Artificial Intelligence
 - Calculus
 - Linear Algebra
 - Probabilities
 - Statistics
 - Machine Learning

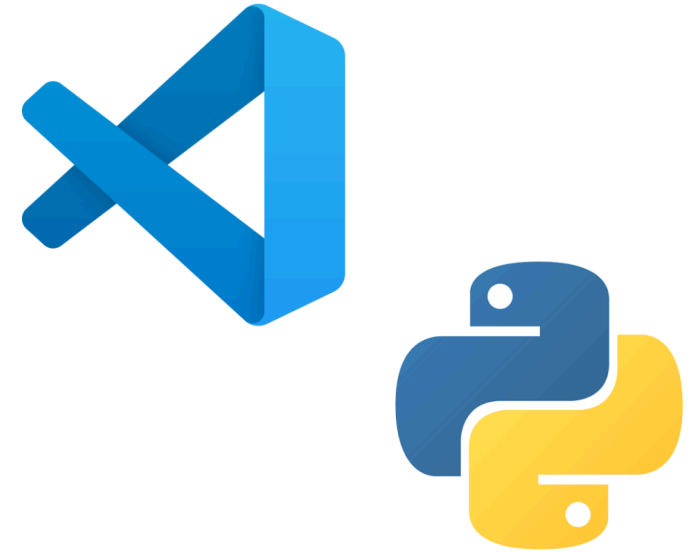
Chapter 2: Getting started!

2.1. Python Installation

To use python in your computer you will need to install two things:

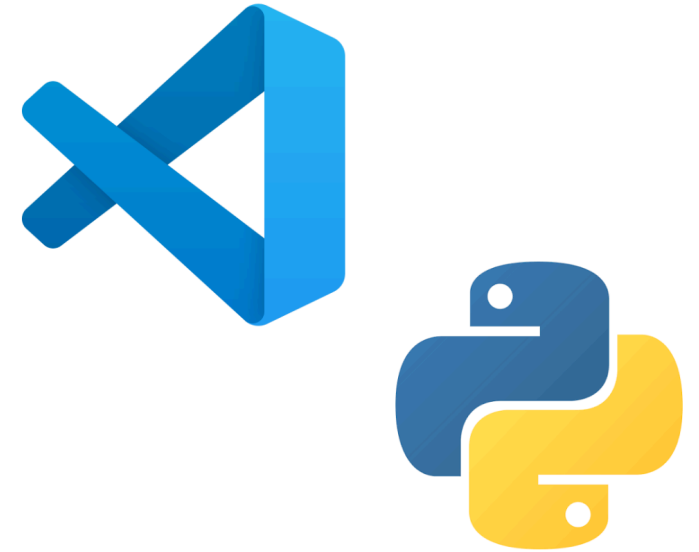
- **A Python interpreter:** To run your python code
- **A text editor:** To write your code. For us, that will be **Visual Studio Code**

We will learn more about what a python interpreter shortly in this chapter.



2.1. Python Installation

1. Install python at the python.org/downloads
2. Install **Visual Studio Code** at the code.visualstudio.com/download
3. Install the **python extension** in VSCode



2.2. Hello, World!

من التقاليد المتبعة عند البدء في تطوير المشاريع البرمجية أن يكون أول برنامج نكتبه هو برنامجًا بسيطًا يعرض الرسالة Hello, world! على الشاشة:

```
print("Hello, python world!")
```

output:

```
Hello, python world!
```

2.2. Hello, World!

لنطلق نظرةً على ما يفعله Python عند تشغيل الملف `hello_world.py`. ينفّذ Python قدرًا معتبرًا من العمليات، حتى عند تشغيل برنامج بسيط كهذا.

عند تشغيل الملف `hello_world.py`، تشير اللاحقة (File Type) `.py` إلى أن هذا الملف هو برنامج مكتوب بلغة Python. بعد ذلك، يقوم المحرر (Editor) بتشغيل الملف باستخدام مفسّر Python (Interpreter)، الذي يقرأ البرنامج ويحدد معنى كل تعليمة فيه. فعلى سبيل المثال، عندما يصادف المفسّر التعليمة `print` متبوعةً بأقواس، فإنه يعرض على الشاشة كل ما يوجد داخل تلك الأقواس.

2.3. The Python Interpreter & REPL

In your terminal where you ran `hello_world.py` , try the following commands:

- `python`
- `python3`
- `py`

إذا نجحت أيُّ منها، فمن المفترض أن يتغيّر موجّه الأوامر (Prompt) في الطرفية إلى شكلٍ مشابه لما يلي:

```
Python 3.14.6 (main, Jun 15 2026, 11:36:54) [GCC 16.1.1 20260430] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

2.3. The Python Interpreter & REPL

That new prompt is called a **REPL**, meaning:

Read	→	قراءة
Evaluate	→	تنفيذ
Print	→	عرض النتيجة
Loop	→	التكرار

يوفّر هذا الوضع طريقةً مباشرة للتفاعل مع مفسّر Python (interpreter) وتجربة مقاطع صغيرة من البرمجة. لואعدنا كتابة مثال `Hello, world!` الخاص بنا:

```
>>> print("Hello, python world!")  
Hello, python world!
```

2.4. Variables and Simple Data Types

You can add (+), subtract (-), multiply (*), and divide (/) **integers** in Python.

```
>>> 2 + 3
5
>>> 3 - 2
1
>>> 2 * 3
6
>>> 3 / 2
1.5
```

Definition

An **integer** is a signed number ((+) او موجبا (-) او سالبا) (عدد صحيح, يَحتمل ان يكون سالبا (-) او موجبا (+))

2.4. Variables and Simple Data Types

تستخدم لغة Python رمزي الضرب ($**$) لتمثيل عملية رفع العدد إلى أس (الأسس).

```
>>> 3 ** 2
9
>>> 3 ** 3
27
>>> 10 ** 6
1000000
```

تدعم لغة Python أيضًا ترتيب العمليات الحسابية، لذا يمكنك استخدام عدة عمليات في تعبير واحد. كما يمكنك استخدام الأقواس لتغيير ترتيب تنفيذ العمليات، بحيث يقوم Python بتقييم التعبير بالترتيب الذي تحدده. على سبيل المثال:

```
>>> 2 + 3*4
14
>>> (2 + 3) * 4
20
```

Chapter 7: Coding a Guessing game

Coding a Guessing game

The program generates a random secret number n in the range $[1, 100]$. The user repeatedly attempts to guess the number. After each incorrect guess, the program informs the user whether the guess is *too low* or *too high*, then prompts for another guess. This process continues until the user correctly guesses the secret number.

يقوم البرنامج بتوليد عددٍ سريٍّ عشوائي n ينتمي الى المجال $[1, 100]$ يحاول المستخدم تخمين هذا العدد بشكل متكرر. بعد كل تخمين غير صحيح، يُخبره البرنامج ما إذا كان التخمين أصغر من العدد المطلوب أو أكبر من العدد المطلوب، ثم يطلب منه إدخال تخمين آخر. تستمر هذه العملية حتى يتمكن المستخدم من تخمين العدد السري بشكل صحيح.

Coding a Guessing game

Example with a secret number $n = 78$:

```
Welcome to the guessing game!
```

```
Enter your guess: 50  
too low!
```

```
Enter your guess: 75  
too low!
```

```
Enter your guess: 81  
too high!
```

```
Enter your guess: 78
```

```
Congratulations, you win!
```